

Chain of Responsibility

Patrones de comportamiento



Alexander Wang Wu C28559



Introducción

El patrón Chain of Responsibility (Cadena de Responsabilidad, CoR, Chain of Command) es un patrón de comportamiento que permite que un objeto transmita una solicitud a través de una cadena de manejadores (handlers).

Cada manejador de la cadena decide si procesa la solicitud o la transmite al siguiente manejador.



Problema común

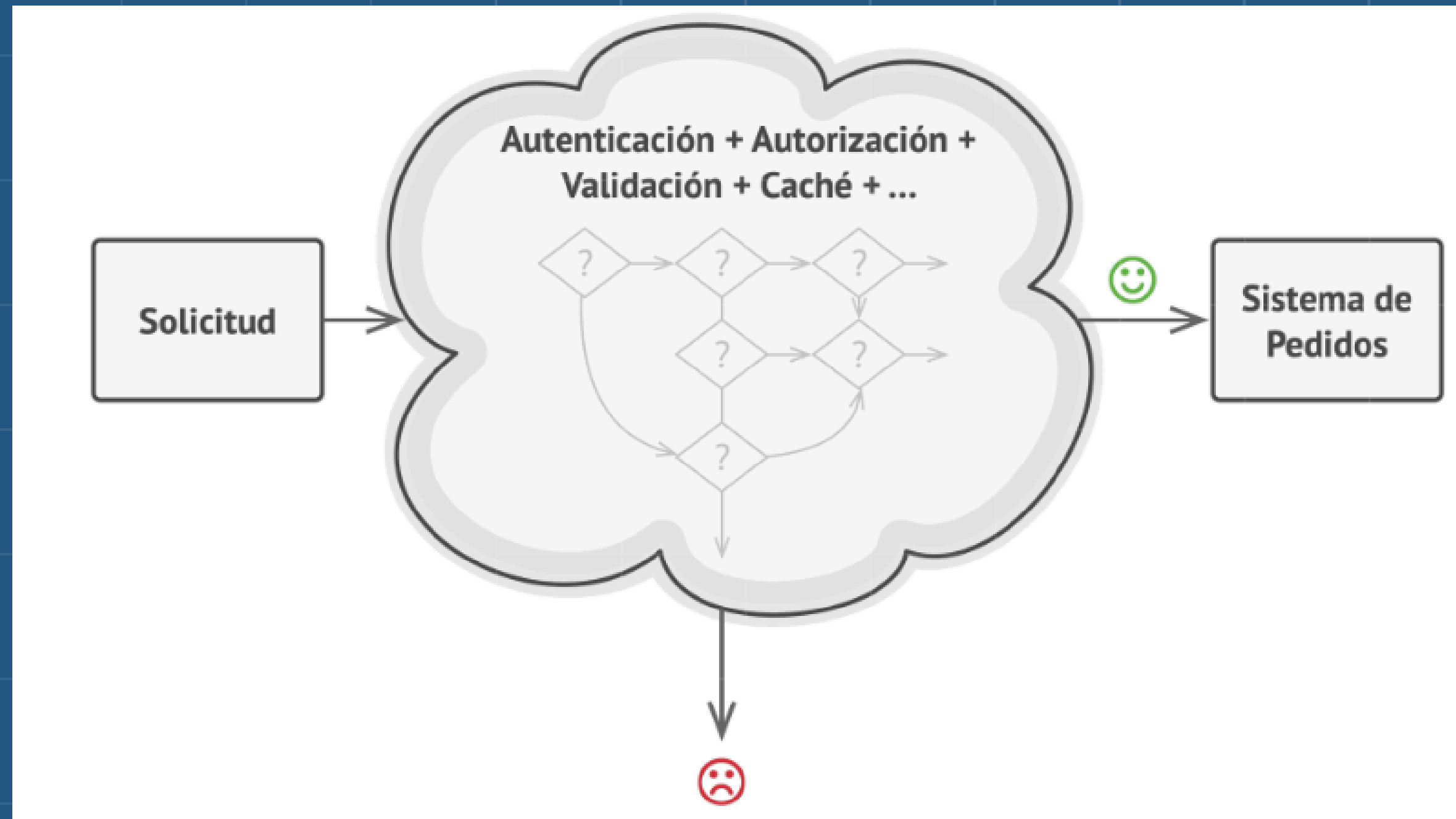


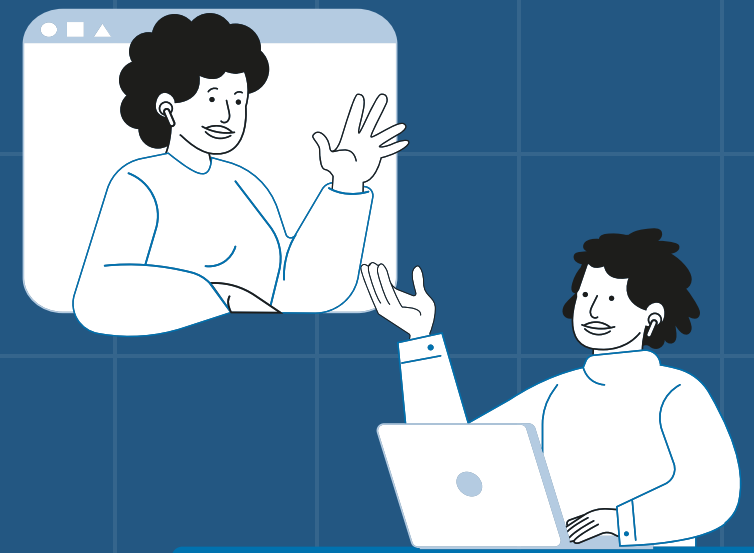
Figura 1. Flujo de solicitud en un sistema de pedidos. <https://refactoring.guru/es/design-patterns/chain-of-responsibility>



Problema común

Problema común que resuelve el CoR es cuando tiene una petición que podría ser atendida por varios componentes/reglas que termina con:

- muchos ifs/switch gigantes anidados.
- baja extensibilidad en donde para agregar una nueva regla/paso debe editar mucho el código de las verificaciones.
- difícil de probar el código porque la lógica está mezclada por muchos elementos.



Solución

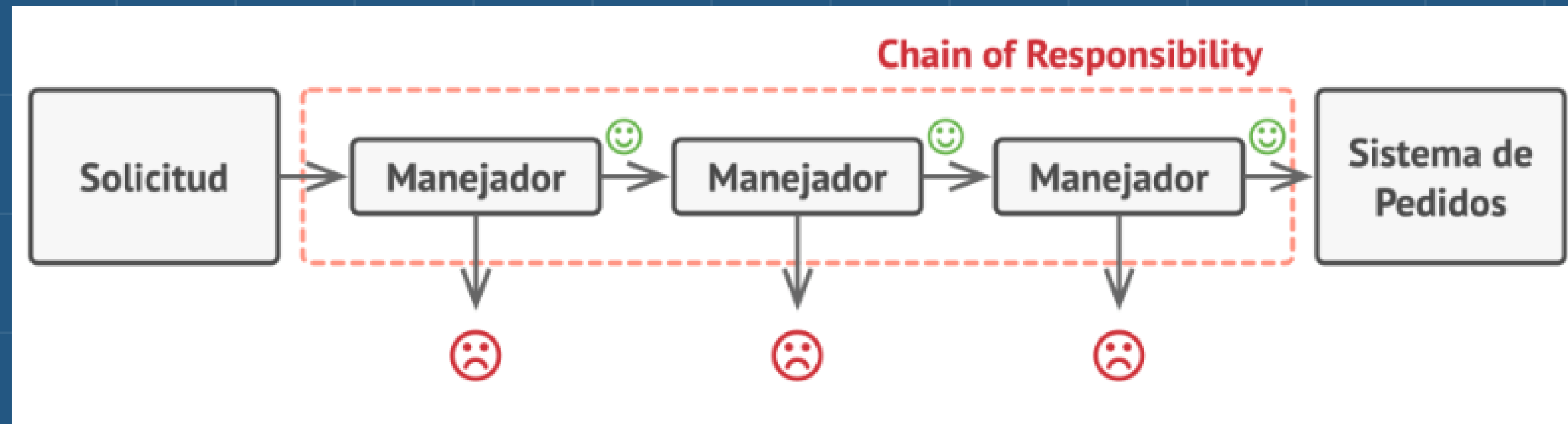
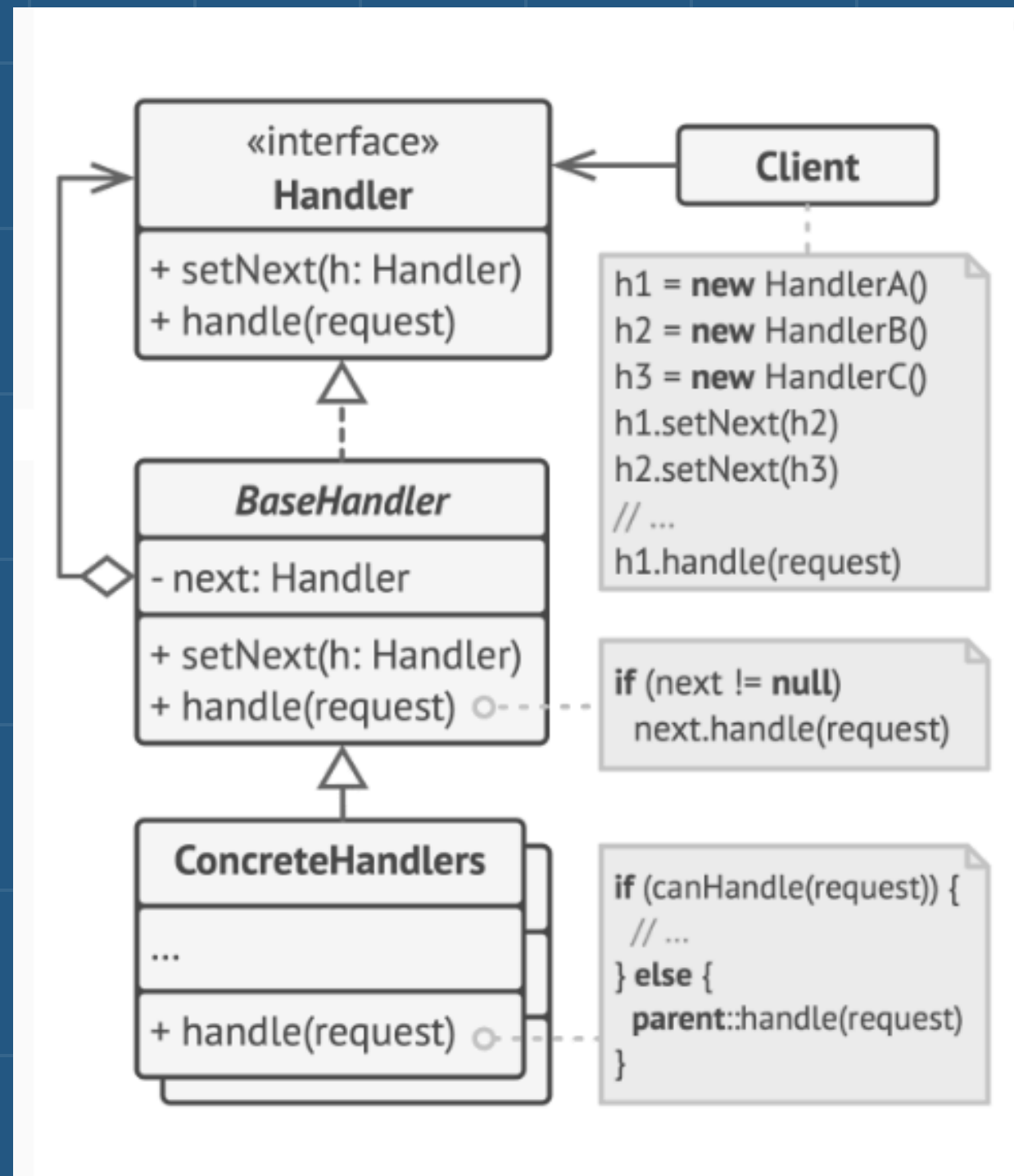


Figura 2. Flujo de solicitud de un sistema de pedidos utilizando CoR. <https://refactoring.guru/es/design-patterns/chain-of-responsibility>



Componentes de la Estructura CoR



1. La clase **Manejadora** declara la interfaz común a todos los manejadores concretos.

2. La clase **Manejadora Base** es opcional y es donde puede colocar el código común para todas las clases manejadoras.

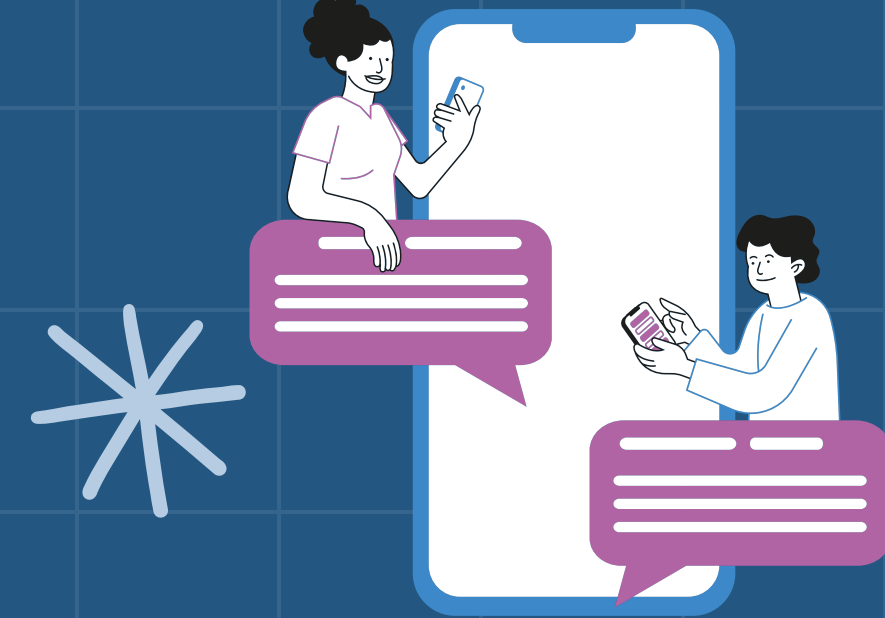
3. Los **Manejadores Concretos** contienen el código para procesar las solicitudes.

4. El **Cliente** puede componer cadenas una sola vez o componerlas dinámicamente, dependiendo de la aplicación.

Figura 3. Estructura de CoR.
<https://refactoring.guru/es/design-patterns/chain-of-responsibility>

Mantenibilidad y Escalabilidad

- Acomplamiento flexible: El remitente de una solicitud no necesita saber qué objeto específico la gestionará.
- Cadena dinámica: Mientras el programa se ejecuta, modificar la cadena es sencillo, esto le otorga a su código una gran flexibilidad.
- Orden secuencial: Las solicitudes se mueven por la cadena una a una, teniendo la oportunidad de procesar la solicitud cada gestor en un orden específico.
- Mecanismo de respaldo: Esto significa que existe una forma predeterminada de gestionar las solicitudes que no encajan en ningún otro lugar.



Ventajas

- Reducción de if/switch enormes.
- Principio abierto/cerrado (Open/Closed).
- Principio de Responsabilidad Única (SRP).
- Controlar el orden de control de solicitudes.



Desventajas

- Rendimiento: puede generar sobrecarga y latencia al tener una cadena larga.
- Complejidad: puede ser difícil de depurar.
- Algunas solicitudes pueden acabar sin ser gestionadas.



Relaciones de patrones con CoR

- Command, Mediator y Observer para conectar emisores y receptores
- Se utiliza a menudo junto Composite
- Los manejadores de CoR pueden implemetar como Command
- CoR y Decorator tienen estructuras de clase muy similares





Ver código del ejemplo CoR





Conclusión



Los patrones de diseño tienen sus ventajas y desventajas, por eso se pueden combinar diferentes patrones para realizar un mismo programa.



Referencias



GeeksforGeeks. (2025). Chain of responsibility design pattern: System design.
<https://www.geeksforgeeks.org/system-design/chain-responsibility-design-pattern/#how-to-implement-chain-of-responsibility-design-pattern>

Refactoring.Guru. (s. f.). Cadena de responsabilidad (Chain of Responsibility).
<https://refactoring.guru/es/design-patterns/chain-of-responsibility>



¡Muchas gracias!

